

Stat 계층 리팩터 — Health 추출 God 컴포넌트 분해

캐릭터·몬스터의 도메인 전체를 떠안던 Stat 계층에서 HP·피격·죽음·무적 책임을 단일 Health 컴포넌트로 분리한 과정과 그 전/후를 비교한다. 빅뱅 없이 6개 증분 스텝으로, 각 단계마다 컴파일 + 전투 1회를 검증하며 진행했다.

10→1

Stat 책임
10가지 → 스탯 합산만

15곳

소비 지정
facade로 무중단 이관

6

증분 스텝
고위험은 단 1개

0

Health 서브클래스
분기=이벤트 구독

5/6

데미지 경로
인터페이스라 무수정

● BEFORE — God 컴포넌트

CharacterStat이 스탯 저장·장비·에너지·쿨타임·피격·죽음·크리·세이브·음성·데미지 UI를 한 클래스에 떠안았고, MonsterStat.HandleDeath는 경제 로직(경험치·드랍·UI)까지 수행. HP와 전투가 스탯 그릇에 강결합.

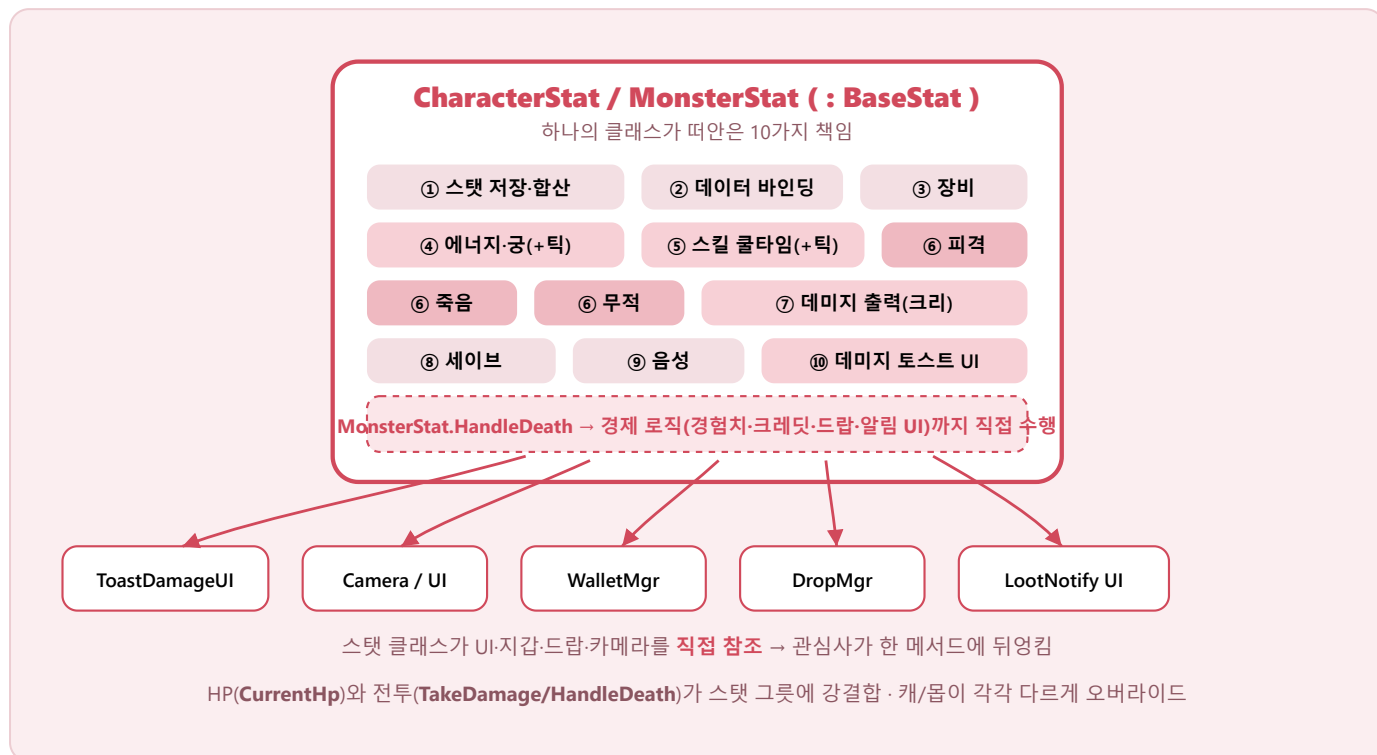
● AFTER — 컴포지션

BaseStat = 순수 StatCollection(= GAS AttributeSet). Health가 HP·피격·죽음·무적 + IDamageable을 소유. 캐/몹 죽음의 차이는 서브클래스가 아니라 이벤트 구독자가 흡수.

핵심 서사. "준-빅뱅"(15곳 동시 수정 + 캐/몹 죽음 로직 상이) 우려로 한때 보류했던 작업을, ① BaseStat 전환기 facade + ② 분기의 이벤트화 두 장치로 동시성을 제거해 안전한 증분으로 완료했다.

❌ 문제 — 책임 과부하

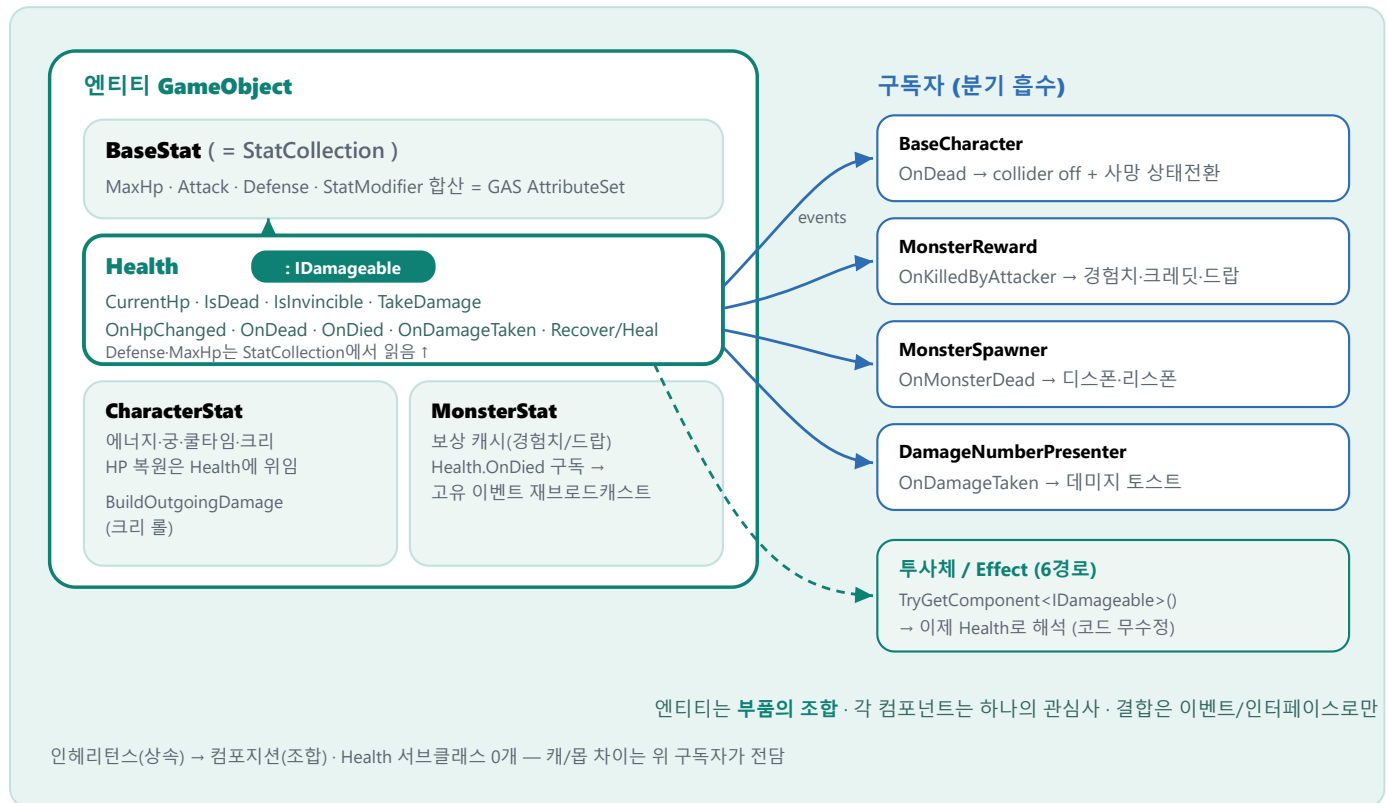
BaseStat이 "스탯 그릇 + 전투(피격/죽음) + UI"를 한꺼번에 갖고, 서브클래스가 그 위에 도메인 전체를 엮었다. 스탯 클래스가 지갑·드랍·카메라·토스트 UI를 직접 참조한다.



Health로 빠질 책임(⑥) 다른 컴포넌트 후보(④⑤⑦⑩) Stat 잔존(①②③⑧⑨)

진단. 1·2단계에서 경제(→MonsterReward)와 토스트 UI(→DamageNumberPresenter)는 이미 추출됐다. 남은 가장 무거운 덩어리가 ⑥ HP·피격·죽음·무적 — 이번 3단계의 대상.

한 GameObject에 역할별 컴포넌트를 조합한다. Health는 방어자측(HP/죽음/무적)만, BaseStat은 공격력·방어력·최대HP 같은 속성 합산만. 죽음의 캐/몹 차이는 이벤트 구독자가 처리한다.



방어자측 상태(Health) 속성/도메인(Stat) 이벤트 구독자 IDamageable seam

≡ 책임 이동 — 무엇이 어디로 갔나

책임	BEFORE (한 클래스)	AFTER (소유자)	방식
현재 HP / 죽음 플래그	BaseStat.CurrentHp / IsDead	Health.CurrentHp / IsDead	이동
피격 계산	Char/MonsterStat.TakeDamage (각각 오버라이드)	Health.TakeDamage (단일 구현)	통합
죽음 처리	HandleDeath (분기)	Health.Die → OnDead/OnDied	이벤트화
무적	CharacterStat.IsInvincible	Health.IsInvincible	이동
데미지 수신 인터페이스	BaseStat : IDamageable	Health : IDamageable	seam
죽음 → 경제	MonsterStat.HandleDeath 내부	MonsterReward (OnDied 구독)	구독
죽음 → 리스폰	MonsterStat.HandleDeath 내부	MonsterSpawner (재브로드캐스트)	구독
사망 → collider off	CharacterStat.HandleDeath	BaseCharacter (OnDead 구독)	구독
데미지 토스트	BaseStat.OnDamageTaken	Health.OnDamageTaken	이동
MaxHp / 공격 / 방어	BaseStat	BaseStat (유지)	유지
크리 데미지 조립	CharacterStat.BuildOutgoing	CharacterStat (유지·공격자측)	유지

BaseStat — Before

스탯 + HP상태 + 피격 + 죽음 + 무적 + 데미지 UI 이벤트 + IDamageable을 한 클래스가 노출. 서브클래스가 TakeDamage/HandleDeath를 제각각 오버라이드.

BaseStat — After

순수 StatCollection: MaxHp/Attack/Defense, GetStat, modifier 구독, BuildOutgoingDamage, 그리고 Health로의 접근점 HealthComp만. HP 상태는 노출하지 않음.

단일 Health(서브클래스 0개) 결정. 죽음 사이드이펙트가 이미 이벤트로 빠져 Health 내부에 override 지점이 없다 → 서브클래스를 만들면 빈 겹데기 3개가 됐을 것. 무적·풀가드는 몬스터에 무해한 기본값 필드로 균일화. (README §3-1 "값이 다르면 asset, 로직이 다르면 클래스"의 정확한 적용)

⚙️ 준-빅뱅을 없앤 2개 안전장치 + 6스텝

장치 ① BaseStat 전환기 facade

상태를 Health로 옮기되 BaseStat.CurrentHp/IsDead/OnHpChanged를 HealthComp로 위임하는 얇은 멤버로 남긴다. 소비 15곳이 계속 컴파일 → 동시 수정 소멸. 마지막에 facade를 지우면 컴파일 에러가 누락 소비자를 색출(안전망).

장치 ② 분기를 이벤트로

캐/몹 죽음의 차이를 서브클래스가 아니라 Health.OnDead/OnDied 구독자가 처리. Health는 단일 균일 컴포넌트로 유지. "안 만드는 결정".

6개 증분 스텝 — 각 단계 컴파일 + 전투 1회 검증

스텝	작업	파일	위험
1	Health 생성(HP·사망 소유) + BaseStat facade화. 자기부트스트랩 lazy HealthComp getter로 부착 타임 NRE 차단	2	저
2	데미지·죽음 → Health.TakeDamage 통합. Stat의 TakeDamage/HandleDeath 삭제, 분기를 구독자로. 무적·OnDamageTaken도 이관. _isDead→Health.IsDead 통일	6	고 (유일)
3	(2에 흡수 — 무적 가드가 TakeDamage에 필요)	—	—
4	IDamageable seam을 Stat→Health로 원자적 이관(이중구현 순간 금지). 투사체 6경로는 인터페이스라 무수정, typed 호출 1곳만 수정	5	중 (격리)
5	소비 15곳을 facade→stat.HealthComp.*로 클러스터 이관(UI·파티·세이프). facade는 유지(안전망)	15	기계적
6	facade 전면 삭제 → BaseStat=순수 StatCollection. CharacterStat 내부 HP대입·Recover/Heal 재배선	7	정리

1

골격+facade

2

데미지·죽음 통합

4

IDamageable seam

5

소비자 이관

6

facade 삭제

롤백 앵커. 착수 전 checkpoint/pre-health-refactor 태그(커밋 7bde3e21)를 찍어, 이상 시 `git reset --hard` 한 방으로 전체 복귀 가능하게 했다.

{} 코드 전/후 — 피격·죽음

캐릭터와 몬스터가 제각각 오버라이드하던 TakeDamage/HandleDeath가 단일 Health.TakeDamage로 통합됐다.

BEFORE — 2벌의 분기 구현

```
// CharacterStat.cs
public override void TakeDamage(DamageInfo d){
    if (IsInvincible || IsDead) return;
    var dmg = Max(d.Amount - Defense, 1);
    CurrentHp -= dmg; Clamp(...);
    RaiseDamageTaken(...);
    if (CurrentHp ≤ 0) HandleDeath(d.Attacker);
    CallOnHpChanged(...);
}

protected override void HandleDeath(...){
    IsDead = true;
    collider.enabled = false; // 캐릭터
    CallOnDead();
}

// MonsterStat.cs - 또 다른 구현
public override void TakeDamage(...){ ... }
protected override void HandleDeath(...){
    OnKilledByAttacker?.Invoke(shooter); // 경제
    CallOnDead();
    OnMonsterDead?.Invoke(_controller);
}
```

AFTER — 단일 구현 + 구독자

```
// Health.cs - 캐/몬 공용 단일 경로
public void TakeDamage(DamageInfo d){
    if (IsInvincible || IsDead) return;
    var dmg = Max(d.Amount - DefenseValue, 1);
    CurrentHp = Clamp(CurrentHp - dmg, 0, MaxHpValue);
    OnDamageTaken?.Invoke(...);
    if (CurrentHp ≤ 0) Die(d.Attacker);
    OnHpChanged?.Invoke(...);
}

void Die(GameObject atk){
    IsDead = true;
    OnDied?.Invoke(atk); // 공격자 전달
    OnDead?.Invoke();
}

// 차이는 '구독자'가 흡수 —————
// BaseCharacter : OnDead → collider off
// MonsterReward : OnDied → 경제
// MonsterStat : OnDied → OnMonsterDead 재발행
```

IDamageable seam — 데미지 6경로가 코드 변경 없이 Health로

```
// 투사체·Effect (Bullet/DealDamage/AreaStrike/
// Lightning/GenesisAttack) - 변경 0줄
if (other.TryGetComponent<IDamageable>(out var t))
    t.TakeDamage(info);
// Before: t == BaseStat → After: t == Health
// (BaseStat/Char/MonsterStat에서 IDamageable
// 선언 제거 + Health에 추가 = 원자적 스왑)
```

데미지 6경로 중 5경로가 이미

TryGetComponent<IDamageable>. Health가 이 인터페이스를 물려받는 순간 투사체·Effect 코드는 한 줄도 안 바뀌었다. typed 직접 호출은 보스 미니언 즉사 1곳뿐 → stat.HealthComp.TakeDamage로만 수정.

1

컴포넌트당 관심사
(SRP 달성)

0

투사체 데미지
코드 변경 줄

6→1

TakeDamage 구현
2별 오버라이드 제거

4

죽음 구독자
(분기 흡수)

✓

중복 필드
_isDead 통일

얻은 것

- **SRP** — Stat=속성, Health=생명, 컨트롤러=행동으로 관심사 분리
- **확장성** — 새 데미지원은 IDamageable만 알면 됨
- **GAS** 정합 — BaseStat이 AttributeSet, Health가 그 위 파생 값
- **버그 정리** — 오래 미뤘던 _isDead vs IsDead 중복 해소
- **무중단** — 6스텝 전 구간 각각 전투 검증 통과

핵심 설계 결정

- **단일 Health** — 분기가 이벤트로 빠져 서브클래스 불필요
- **IDamageable = seam** — 인터페이스가 이미 이음매라 스왑이 투명
- **재브로드캐스트** — MonsterStat이 Health.OnDied를 되쏘아
Spawner/Reward 무수정
- **BaseStat→StatCollection 리네임 스킵** — 참조 churn 대비 변별력 낮아
"안 하는 결정"

배운 점 — 큰 리팩터를 안전한 증분으로 쪼개는 법

1. **전환기 facade로 동시성 제거.** 소비자를 한꺼번에 고치지 말고, 위임 껍데기를 남겨 한 클러스터씩 옮긴다.
2. **삭제를 안전망으로.** 마지막에 facade를 지우면 컴파일러가 누락을 대신 찾아 준다.
3. **위험을 한 스텝에 격리.** 고위험(피격-죽음 통합)은 Step 2 하나로 몰고, seam 이 동은 Step 4로 분리.
4. **이음매를 먼저 찾는다.** 이미 인터페이스로 추상화된 지점(IDamageable)이 무 비용 교체 지점.

후속. StatRefactor 4단계(에너지/쿨타임 → EnergyGauge 분리)는 동일 패턴으로 필요 시 진행. Health 위에는 RoleSystem의 버프/방깍(Attribute 양방향 합산)이 없힌다.